

Source3: A Rule-Based Pipeline for Equation Annotation in Scientific Papers

1 Overview

1.1 Problem Analysis

The task is to build a generally applicable system for quantum physics papers from arXiv. For every paper in the assigned list, the system must identify the important numbered equations, extract their meaning, explain the symbols and abbreviations used in them, and describe relations between equation pairs in a graph-ready JSON format. Each output entry also needs an audit trail so that the extracted information can be traced back to the source document.

Several approaches are possible. A purely rule-based system is transparent and easy to audit, but brittle when papers use different notation or wording. A retrieval-based system using BM25 or dense scientific embeddings can find relevant explanatory sentences more flexibly, but still needs filters to avoid weak matches. Generative language models could summarise equations, but they are less suitable here because the dataset requires traceable evidence from the paper. Source3 therefore combines deterministic document parsing, lexical and semantic retrieval, and hand-written extraction patterns to keep the prototype both general enough for arbitrary papers and auditable enough for the required dataset.

1.2 Approach Overview

Source3 is a deterministic, non-generative pipeline built around a shared embedding space. After downloading and parsing the arXiv HTML, the system extracts numbered equations, nearby sentences, cross-references, sections, and MathML symbol structure. Equations and text chunks are then embedded with MathBERT [1] (witiko/mathberta), so that mathematical expressions and their surrounding explanations can be compared by cosine similarity.

The same embedding representation is used for the three main extraction tasks.

- **Equation meaning:** the equation and its context are matched against candidate explanatory sentences from the paper.
- **Symbol meaning:** each symbol is used as a query to retrieve likely definition sentences, which are then checked with simple linguistic patterns such as “where x is” or “ x denotes”.
- **Relations:** pairs of equations are compared through their embedded contexts, shared symbols, section distance, and explicit cue phrases such as references to earlier equations or derivation statements.

This design keeps the system general enough for different quantum physics papers while remaining auditable. Embeddings provide flexible retrieval, and deterministic rules select only source-backed spans and record the evidence used in the final JSON.

1.3 Pipeline Diagram

The diagram below shows the data flow through the ten stages.



2 System Architecture

This section briefly describes how the pipeline prepares the paper data for extraction. The examples use paper 2401.00059.

2.1 Stage 2: Document Parsing

Responsible file: source3/stage02_build_documents.py

Output: data/pipeline/documents/<paper_id>.json.

The parser reads the cached arXiv HTML and stores a structured document: sections, paragraphs, sentences, equation blocks, MathML, equation labels, cross-references, and before/after equation context.

```
{
  "paper_id": "2401.00059",
  "sentences": {
    "sec:section:2:p2:s6": {"text": "We write down ..."}
  },
  "raw_equations": [{
    "raw_equation_id": "raw:0001",
    "anchor_id": "S2.E1",
    "visible_labels": ["1"],
    "latex": "\\ddot{u}_{\\{n\\}}=-\\beta(u_{\\{n\\}}-u_{\\{n-1\\}})-..."
  }]
}
```

2.2 Stage 3: Chunk Construction

Responsible file: source3/stage03_build_chunks.py

Output: data/pipeline/chunks/<paper_id>.json.

The chunk builder reads the Stage 2 document and creates searchable units: sentence, paragraph, section context, raw equation, equation-neighbourhood, and cross-reference chunks. Each chunk keeps its source IDs and document position.

```
{
  "chunk_id": "2401.00059:raw_equation_neighborhood:raw:0001",
  "chunk_type": "raw_equation_neighborhood",
  "raw_equation_anchor": "S2.E1",
  "source_ids": {
    "raw_equation_id": "raw:0001",
    "before_sentence_ids": ["sec:section:2:p2:s6"],
    "after_sentence_ids": ["sec:section:2:p3:s1"]
  }
}
```

2.3 Stage 4: Equation Extraction

Responsible file: source3/stage04_extract_equations.py

Output: data/pipeline/equations/<paper_id>.json.

This stage selects the numbered display equations for the dataset. For each equation it stores the final equation ID, normalized $\text{\LaTeX}$, raw equation ID, HTML anchor, section, document order, and resolved before/after context sentences.

```
{
  "paper_id": "2401.00059",
  "equations": [{
    "equation_id": "1",
    "latex": "\\ddot{u}_n=-\\beta(u_n-u_{n-1})-...",
    "raw_equation_id": "raw:0001",
    "anchor_id": "S2.E1",
    "before_sentences": [{"sentence_id": "sec:section:2:p2:s6"}]
  }]
}
```

2.4 Stage 5: Embedding Construction

Responsible file: source3/stage05_build_embeddings.py

Output: data/pipeline/embeddings/<paper_id>.json
and data/pipeline/embeddings/<paper_id>.npz.

This stage embeds equations and text chunks with MathBERT. For each equation it builds equation-only, summary-context, and sentence-context vectors. For the paper text it embeds the retrieval chunks from Stage 3. The vectors are saved in .npz; the JSON file records row metadata and source text for auditing.

```
{
  "paper_id": "2401.00059",
  "model": "witiko/mathberta",
  "normalized": true,
  "dimension": 768,
  "rows": [{
    "vector_id": "2401.00059:equation:1:summary",
    "vector_kind": "summary",
    "source_text": "before: ... equation: ... after: ...",
    "row": 1
  }]
}
```

3 Annotation and Export

After Stage 5, the pipeline has structured equations, retrieval chunks, and embedding vectors. Stages 6–10 use these artifacts to create the required dataset fields: equation meaning, symbols, symbol definitions, equation relations, and the final audit-ready JSON.

3.1 Stage 6: Equation Meaning Extraction

Responsible file: source3/stage06_extract_meanings.py

Output: data/pipeline/meanings/<paper_id>.json.

This stage finds one source-backed meaning sentence for each selected equation. It first builds a candidate pool from cross-reference chunks, cue phrases around the equation, BM25 retrieval over sentence and equation-neighbourhood chunks, and finally a proximity fallback.

BM25 role. BM25 is used as the lexical retrieval component. For each equation, the query is built from the equation $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, the equation label, and the two nearest preceding context sentences. The

system tokenizes this query and every candidate chunk, then scores chunks by term overlap using BM25 with length normalization. For example, for paper 2401.00059, equation (1), the BM25 query is:

```
\ddot{u}_{n}=-\beta(u_{n}-u_{n-1})-\beta(u_{n}-u_{n+1})
Equation 1
We denote the displacement of the atom on the left side as  $u^L_n$ 
and on the right side as  $u^R_n$ .
For all the atoms but those on the interface ( $u^L_0$  and  $u^R_0$ )
we write down the motion equation in the form
```

This favors sentences that share rare and informative words or symbols with the equation context, while avoiding a bias toward very long paragraphs. The top BM25 chunks are added as possible meaning candidates.

MathBERT role. MathBERT is used for semantic comparison. Stage 5 already stores a *summary vector* for each equation, created from the equation plus its surrounding context, and vectors for sentence or neighbourhood chunks. In Stage 6, the cosine similarity between the equation summary vector and each candidate chunk vector estimates whether the candidate is semantically close to the equation, even when it does not repeat the same words. For the same example, the MathBERT summary input is:

```
before: We denote the displacement of the atom on the left side as  $u^L_n$ 
and on the right side as  $u^R_n$ . For all the atoms but those on the
interface ... we write down the motion equation in the form
equation: \ddot{u}_{n}=-\beta(u_{n}-u_{n-1})-\beta(u_{n}-u_{n+1})
after: where we omitted the side index L or R, since the equation is the
same for both indexes.
```

Final ranking. The final score combines four signals: MathBERT cosine similarity, BM25 score, source reliability (cross-references are trusted more than proximity fallback), and distance from the equation. Candidates that look procedural, math-heavy, incomplete, or more like a symbol definition receive a penalty. The highest-scoring remaining candidate is written as the equation meaning, together with an audit record describing how it was selected.

3.2 Stage 7: Symbol Extraction

Responsible file: source3/stage07_extract_symbols.py

Output: data/pipeline/symbols/<paper_id>.json.

This stage extracts the mathematical symbols that occur in each equation. The main source is the MathML tree, because MathML preserves structure such as subscripts, superscripts, and decorated symbols. If MathML is missing or incomplete, the system falls back to parsing the $\LaTeX$ string. The output keeps canonical symbol names together with their possible $\LaTeX$ forms, so later stages can search for definitions using several aliases.

3.3 Stage 8: Symbol Definition Lookup

Responsible file: source3/stage08_extract_symbol_meanings.py

Output: data/pipeline/symbol_meanings/<paper_id>.json.

This stage searches for the meaning of each extracted symbol. For every symbol, the system first builds a set of aliases from its canonical name, base symbol, Unicode form, and $\LaTeX$ forms. For example, one symbol may be searchable as beta, $\backslash\beta$, and β . The query is then phrased around common

definition language, such as `where beta is` and `beta denotes`, and embedded with MathBERT. For example, for the extracted symbol `beta` from paper 2401.00059, the query is:

```
where beta is beta denotes beta beta_unicode \beta
```

The embedded query is compared against the sentence-chunk vectors from Stage 5. This comparison uses cosine similarity: the query vector and the stored sentence vectors are normalized, so their dot product gives the cosine score. The top 30 most similar sentences are kept as candidates. This semantic pre-filter is important because a symbol definition may be several sentences away from the equation and may not occur in the immediate local context.

After retrieval, deterministic patterns extract the actual definition span. The system checks patterns in priority order:

- **where-is:** “where x is ...”;
- **denotes:** “ x denotes ...”;
- **let-be:** “let x be ...”;
- **reverse noun phrase:** “the coupling constant x ”;
- **respectively:** matches symbol lists to definition lists.

Candidate definitions are rejected if they are too long, too short, math-dominated, or copied directly from the equation. The important design choice is conservative extraction: if no reliable source-backed definition span is found, the symbol is left undefined rather than filled with a guessed meaning.

3.4 Stage 9: Relation Detection

Responsible file: `source3/stage09_build_relations.py`

Output: `data/pipeline/relations/<paper_id>.json`.

This stage builds graph-ready directed relations between equation pairs within the same paper. For every ordered pair of selected equations (A, B) , the system checks whether the context around equation A mentions equation B . It first searches the before/after sentences for equation-number references and cue phrases. Derivation cues include words such as “from”, “using”, “substituting”, and “derived from”. Equivalence cues include “equivalent to”, “reduces to”, and “recovers”. Special-case cues include “special case”, “setting”, and “limit”. If such a cue is found, the relation is marked as *strong*.

If no explicit cue is found, the system uses weaker signals. It computes symbol overlap with the Jaccard score between the two canonical symbol sets. It also checks whether the equations are in the same or neighbouring sections, and compares their MathBERT summary vectors from Stage 5. These signals are useful because related equations often share notation or appear in the same derivation block even when the paper does not explicitly say “Equation (A) follows from Equation (B)”.

The output relation grade reflects the evidence strength. Explicit citations or cue phrases produce *strong* relations. Shared symbols, same-section context, or high semantic similarity produce *potential* relations. If none of the signals fires, the relation is marked *none*.

3.5 Stage 10: Final Export

Responsible file: `source3/stage10_export_final.py`

Output: `data/final_data.json`.

The final stage joins all per-paper artifacts into the required JSON schema. For each equation it writes the equation $\text{\LaTeX}$, extracted meaning, symbol-definition dictionary, relation dictionary, and audit trail. Papers without selected numbered equations are still included with an empty dictionary. The export stage also validates that required fields are present and that relations cover the other selected equations in the same paper.

4 Generalization

The approach generalizes because it uses common arXiv and LaTeXML structure: numbered equations, MathML or $\text{\LaTeX}$ equation source, section and paragraph layout, and references such as “Eq. (1)”. The embedding step improves robustness because it compares equation contexts and candidate explanations semantically, not only by exact word overlap.

The final dataset provides evidence for this. It contains keys for all 100 papers in the assigned list. Among them, 54 papers contain extracted equations and 46 papers are preserved with empty equation dictionaries. The final dataset contains 353 equations, satisfying the requirement to collect at least 350 equations while finishing the last processed paper. Every non-empty equation entry contains the required fields and an audit trail.

The main limits are papers with unusual HTML conversion, missing visible equation labels, implicit symbol definitions, or notation introduced by convention rather than by explicit prose.

5 Quality Discussion

5.1 Strengths

The pipeline creates a complete, traceable dataset. It extracts 353 equation entries; all 353 have non-empty meanings and all 353 include audit trails. It also produces 2048 relation entries with *strong*, *potential*, and *none* grades, which makes the output usable in a graph-ready format.

5.2 Weaknesses and Limits

Symbol definition extraction is the weakest part. The analysis report found 1897 extracted symbols, but only 202 received definitions. That leaves 1695 undefined symbols, or 89.35% empty symbol definitions. The main reason is that many papers do not contain a reliable extractive definition pattern such as “where x is” or “ x denotes”.

Equation meanings are more complete but still need critical review. All 353 meanings are non-empty, but 126 were flagged by postprocessing. Typical problems are incomplete context, symbol-definition sentences selected as equation meanings, math-heavy spans, or phrases that are too short or too long.

5.3 Overall Assessment

The approach is strong for auditable dataset construction because it prefers source-backed evidence over generated explanations. Its main weakness is completeness of symbol definitions, which is limited by how explicitly the source papers define their notation.

6 Conclusion

Source3 builds the required dataset by combining structured arXiv parsing, embedding-based retrieval, deterministic patterns, and audit validation. The embedding space makes the method flexible across different quantum physics writing styles, while the rule-based checks keep outputs traceable. The remaining limitations mainly come from ambiguity in the source papers and from the extractive-only requirement.

References

- [1] S. Peng, K. Yuan, L. Gao, and Z. Tang, “MathBERT: A Pre-trained Model for Mathematical Formula Understanding,” *arXiv preprint arXiv:2105.00377*, 2021.